

The Function Design Process

Lecture 16

CS 1001

Kirk Duran

Today

- ▶ Use a six-step process to move from a problem statement to a tested function.
- ▶ Identify the input and output data before writing implementation code.
- ▶ State the function's purpose precisely enough to determine expected behavior.
- ▶ Write a typed function header that exposes the intended interface.
- ▶ Derive tests from the specification before implementing the body.
- ▶ Implement, run tests, inspect failures, and iterate until the behavior matches the specification.

Key idea

The function body is not the starting point. A disciplined design process determines what the function must do before deciding how it will do it.

Part 1: Why Use a Function Design Process?

Coding Immediately Often Mixes Several Problems Together

Prompt

Write a function that determines whether a Fahrenheit temperature is freezing.

Immediate coding

```
def freezing(temp):  
    ...
```

Questions still unanswered

What input type is expected?
What value should be returned?
Is exactly 32 freezing?
How will we verify the boundary?

Key idea

Implementation becomes easier when the data and expected behavior have already been made explicit.

The Six-Step Function Design Process

1. Identify the data and types.
2. Write a one-line purpose statement.
3. Write the function header with type hints.
4. Write tests from the expected behavior.
5. Write the function body.
6. Run, inspect, and iterate.



Specification Comes Before Implementation

Definition

A **specification** describes the behavior a function is expected to provide: what data it accepts and what result it should produce.

Two different questions

Specification: What should this function do?

Implementation: How will the function produce that behavior?

Key idea

Tests should be derived from the specification. Otherwise a test may merely confirm what the current implementation already happens to do.

Running Example: Determine Whether a Temperature Is Freezing

Problem statement

Design a function that accepts a temperature in degrees Fahrenheit and returns whether that temperature is at or below the freezing point of water.

Do not implement yet

We will deliberately postpone the function body until the input, output, purpose, interface, and tests are established.

In class

Before continuing, identify the input data, its likely Python type, and the type of the result.

Part 2: Step 1 — Identify Data, Variables, and Types

Begin with the Data Crossing the Function Boundary

Input

Fahrenheit temperature

suggested name: `temperature`

expected type: `float`

Output

whether the temperature is freezing

result type: `bool`

Key idea

The input and output types describe the shape of the computation before any algorithm has been chosen.

Choose Names That Describe the Role of the Data

Weak names

x
num
value

More informative names

temperature
weight
tax_rate

- ▶ A parameter name should communicate the role of the argument supplied by the caller.
- ▶ Good names reduce the amount of interpretation required when reading the function header and tests.

Type Hints Document Intended Data

Example annotations

```
temperature: float  
count: int  
name: str  
eligible: bool
```

Technical distinction

Type hints are annotations. Ordinary Python execution does not generally reject a call merely because an argument disagrees with an annotation.

Key idea

In this course, type hints serve as explicit documentation of the interface and support reasoning about valid inputs and expected outputs.

Multiple Inputs Require Separate Roles

Problem

Return the larger of two integers.

Role	Name	Type
first candidate	a	int
second candidate	b	int
result	—	int

Key idea

Step 1 is complete when the data entering and leaving the function is sufficiently explicit to describe the interface.

Part 3: Steps 2 and 3 — Purpose and Function Header

Step 2: Write a One-Line Purpose Statement

Definition

A **purpose statement** is a concise description of the behavior the function provides to its caller.

Running example

Return True when temperature is at or below 32.0 degrees Fahrenheit; otherwise return False.

Key idea

A useful purpose statement is specific enough that another person can derive expected outputs without reading the implementation.

Purpose Statements Describe Behavior, Not Syntax

Too vague

“Checks the temperature.”

Too implementation-specific

“Uses `<=` to compare the argument with 32.”

Example

“Return whether a Fahrenheit temperature is at or below the freezing point of water.”

Key idea

State observable behavior. Leave implementation choices for Step 5.

Step 3: Derive the Function Header from the Data

Data from Step 1

```
input: temperature: float
output: bool
```

Function header

```
def is_freezing(temperature: float) -> bool:
```

- ▶ The function name summarizes the operation.
- ▶ The parameter names the incoming data.
- ▶ The parameter annotation records the intended input type.
- ▶ The return annotation records the intended result type.

The Header Exposes the Function's Interface

Definition

A function's **interface** is the information a caller needs in order to use the function correctly, without depending on its internal implementation.



Key idea

The caller should be able to reason from the function's name, parameters, documented behavior, and returned result without tracing its body. Returning a different kind of result would violate that intended interface.

Checkpoint: Can the Behavior Be Predicted from the Design?

Current design

```
def is_freezing(temperature: float) -> bool:
```

Purpose: return whether the Fahrenheit temperature is at or below 32.0.

In class

Without writing a body, determine the expected results for 31.0, 32.0, and 33.0. Explain which value is the boundary case.

Key idea

If expected results cannot be determined, the specification is not yet precise enough to support reliable testing.

Part 4: Step 4 — Write Tests Before the Body

Tests Translate the Specification into Executable Expectations

Specification

Return whether temperature is at or below 32.0.

Input	Expected result	Reason
31.0	True	below boundary
32.0	True	exactly at boundary
33.0	False	above boundary

Key idea

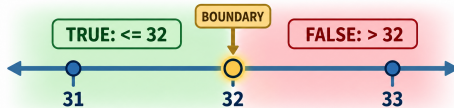
The expected values come from the purpose statement, not from executing the unfinished function.

Boundary Cases Are Determined by the Specification

Decision boundary

`temperature <= 32.0`

- ▶ A value below the boundary exercises one side of the decision.
- ▶ The boundary value checks whether equality is included.
- ▶ A value above the boundary exercises the opposite side.



Write the Test File Before Implementing the Body

test_temperature.py

```
import unittest
from temperature import is_freezing

class TestFreezing(unittest.TestCase):
    def test_below(self):
        self.assertTrue(is_freezing(31.0))
    def test_boundary(self):
        self.assertTrue(is_freezing(32.0))
    def test_above(self):
        self.assertFalse(is_freezing(33.0))
```

Key idea

Before the body is complete, failing tests are expected: they establish the behavior the implementation must eventually satisfy.

Separate Files Separate Responsibilities

`temperature.py`

Defines the behavior being implemented.

`test_temperature.py`

Defines examples of expected behavior.

```
from temperature import is_freezing
```

Key idea

The implementation and its tests evolve together, but they serve different purposes and should remain conceptually distinct.

A Test Should Explain What Behavior It Is Checking

Weak

```
def test1(self):  
    ...
```

Example

```
def test_freezing_point(self):  
    self.assertTrue(  
        is_freezing(32.0))
```

- ▶ The test name should identify the behavior or case under examination.
- ▶ A focused failure message is more informative than a large test containing unrelated expectations.

Tests Must Be Independent of the Implementation Strategy

Acceptable expectation

```
self.assertTrue(is_freezing(32.0))
```

Poor testing idea

“Verify that the function uses an if statement.”

- ▶ The specification requires a Boolean result.
- ▶ It does not require one particular arrangement of statements to obtain that result.

Key idea

Test externally observable behavior unless the specification explicitly requires an implementation property.

Part 5: Step 5 — Write the Function Body

Only Now Do We Implement the Computation

Design already established

```
def is_freezing(temperature: float) -> bool:
```

Implementation

```
def is_freezing(temperature: float) -> bool:  
    return temperature <= 32.0
```

Key idea

The implementation is short because most of the reasoning has already been expressed in the specification, interface, and tests.

Prefer the Simplest Body That Clearly Satisfies the Specification

Direct predicate

```
def is_freezing(t: float) ->  
bool:  
    return t <= 32.0
```

Equivalent conditional

```
def is_freezing(t: float) ->  
bool:  
    if t <= 32.0:  
        return True  
    return False
```

Key idea

Both implementations satisfy the same interface. Prefer the clearer expression of the underlying computation.

Some Specifications Naturally Require Conditional Structure

Problem

Return a shipping cost of 5.0 for packages weighing at most 10.0 pounds; otherwise return 8.0.

Implementation

```
def shipping_cost(weight: float) -> float:  
    if weight <= 10.0:  
        return 5.0  
    return 8.0
```

Key idea

The design process does not prescribe one body shape; the specification determines what behavior must be represented.

Trace the Body Against a Previously Chosen Test

Test case

`shipping_cost(10.0)` should return 5.0.

1. Bind `weight` to 10.0.
2. Evaluate `weight <= 10.0` $\rightarrow$ True.
3. Enter the selected branch.
4. Execute `return 5.0`.
5. Compare actual result 5.0 with expected result 5.0.

Key idea

Tracing remains useful: tests tell us whether behavior matches; tracing explains how a particular result was produced.

Do Not Implement Only the Examples

Incorrect implementation

```
def is_freezing(t: float) -> bool:  
    return t == 31.0 or t == 32.0
```

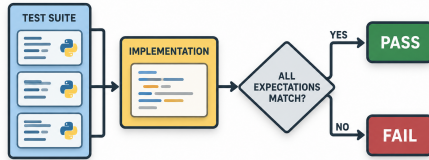
- ▶ This passes two known positive examples.
- ▶ It does not implement the general rule in the specification.
- ▶ For example, `is_freezing(0.0)` would be incorrect.

Key idea

Tests are examples of the specification; they are not a replacement for understanding the general behavior being specified.

Part 6: Step 6 — Run, Inspect, and Iterate

Run the Test Suite After Implementing the Body



Green

Current implementation satisfies the tested expectations.

Red

At least one expectation is not satisfied or a test cannot complete normally.

A Failure Creates a New Reasoning Problem

Suppose the body is

```
def is_freezing(t: float) -> bool:  
    return t < 32.0
```

Observed result

test_below passes.
test_boundary fails.
test_above passes.

In class

What does this failure pattern tell you about the relationship between the implementation and the specification?

Debug the First Meaningful Divergence

1. Start from the failing boundary test.
2. Expected condition result for `32.0`: `True`.
3. Actual condition result for `32.0 < 32.0`: `False`.
4. The divergence occurs in the comparison operator.
5. Correct `<` to `<=`.
6. Run the complete test suite again.

Key idea

Lecture 15's debugging process is now embedded inside the larger function design process.

Iteration Can Improve Either Code or Specification

Implementation problem

The stated behavior is clear, but the body does not produce it.

Specification problem

A reasonable input exposes behavior that the original statement never defined.

Example question

What should `shipping_cost(-2.0)` mean? The original specification may need to state whether negative weights are valid inputs.

Passing Tests Should Remain as Regression Tests

Definition

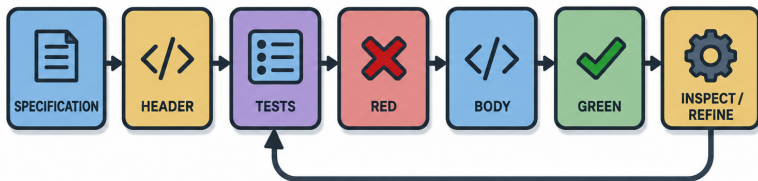
A **regression** occurs when a change causes behavior that previously worked to become incorrect.

- ▶ Keep existing tests when the implementation changes.
- ▶ Add a new test when a newly discovered case should remain supported.
- ▶ Run the whole test suite, not only the test related to the latest edit.

Key idea

A growing test suite preserves executable examples of behavior that future changes should continue to satisfy.

The Complete Development Loop



Worked Example: Return the Larger of Two Integers

1. **Data:** `a: int, b: int; result int.`
2. **Purpose:** return the larger of a and b; if they are equal, return that value.
3. **Header:** `def maximum(a: int, b: int) -> int:`
4. **Tests:**
 - `maximum(5, 3) → 5`
 - `maximum(3, 5) → 5`
 - `maximum(5, 5) → 5`
5. **Body:** `if a >= b: return a otherwise return b`
6. **Iterate:** run all three tests and inspect any mismatch.

Key idea

The equality case is deliberate: it tests the boundary between “first is larger” and “second is larger.”

Integrated Exercise: Design Before Coding

Problem

Design `has_minimum_length(password: str) -> bool`, which returns whether password contains at least eight characters.

In class

Before writing the body, produce:

1. the input and output data;
2. a one-line purpose statement;
3. the complete typed header;
4. three tests that examine the boundary;
5. only then, the simplest correct body.

A Practical Checklist for New Functions

1. What values enter the function, and what are their intended types?
2. What value must the function return?
3. Can the behavior be stated precisely in one sentence?
4. Does the function header reflect that data and behavior?
5. Which ordinary, boundary, and special cases follow from the specification?
6. Do the tests encode those expectations before the body is trusted?
7. Does the simplest clear implementation satisfy the tests?
8. After a correction, did the entire suite still pass?

Takeaways

- ▶ Function design begins with data and behavior, not implementation syntax.
- ▶ Type hints document the intended interface but are not, by themselves, runtime enforcement.
- ▶ Purpose statements make expected behavior explicit enough to test.
- ▶ Tests should be derived from the specification before implementation details influence expectations.
- ▶ The body implements the general behavior; it should not merely memorize the examples in the test suite.
- ▶ Red–green feedback and debugging turn failures into evidence for the next iteration.

Key idea

Design, testing, implementation, and debugging are not separate habits. Together they form a repeatable process for producing functions whose behavior can be explained and verified.